PocketSmart AI

Your Smart Budget & Recommendation Assistant

Project Documentation

Team ID : SWTID-2026-7441

Team Size : 5

Team Leader : Sujitha Marimuthu

Team member : Thirukan V

Team member : Vijay R

Team member : Yogeshwaran P

Team member : Yogeshwaran



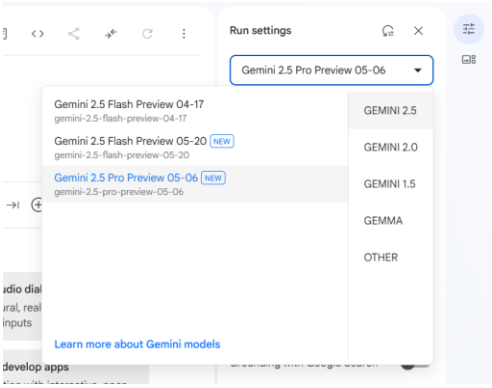
Project Description

dtxddtdtftftfxfdt Managing budgets across differentdiffu68y currentcbcerent life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. PocketSmart AI addresses this challenge through a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services.Using Gemini 1.5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine experience that bridges multiple categories and e-commerce ecosystems.

Scenario 1: HoScenario me

Int erior Planning with Smart Budget Allocation

Users begin by selecting the Home Budget Planner. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quanActivity 3.4: Startup and main function/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.



Project Description

Managing budgets across differentdiffu68y currentcbcerent life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. PocketSmart AI addresses this challenge through a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services.Using Gemini 1.5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine experience that bridges multiple categories and e-commerce ecosystems.

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the Home Budget Planner. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) antities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon.Recommendations are balanced across functionalitytfunctionality, style, and price, ensuring users can decorate their spaces beautifully

without overspending.

Scenario 2: AI-Based Party Budget Planning

In the Party Planner, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across catering, decoration, and entertainment, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO.The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

Scenario 3: Jewelry Recommendations for Occasions

The Jewelry Budget Planner aallowsllows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style.This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

SMARTBRIDGE
Let's Bridge the Gap

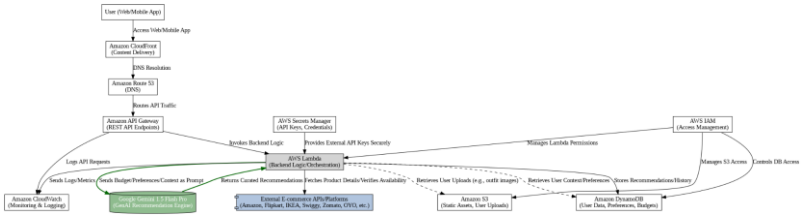
Smart Internz

(In this project we are gonna see about home,party and jewelry budget planner and ai gives recommendation)
Act as an experienced Generative AI Project Developer and carefully analyze the provided project documentation.
Build the complete project from scratch, including all required folders, files, dependencies, and configurations.
Generate full, working code for every file, not just the partial snippets provided in the documentation.
Ensure all frontend, backend, APIs, AI/LLM integration, and configurations work together without errors.

Finally, provide simple VS Code setup, installation, running, and testing instructions for the complete application.
. Architecture Overview
vPocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google's Gemini 1.5 Flash Pro, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.9
Core Technologies
+Flask: Backend service layer for request handling, routing, and session control.
Gemini 1.5 Flash Pro: AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.): Product and service data sourcing.
Frontend (HTML/CSS + JS): Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component
Description
Frontend UI
Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.
Flask Backenrrrrrd
Manages user sessions, routes (/generate-home, /generate-party, /generate-jewelry), and handles communication between UI and Gemini AI.
Gemini AI Layer
Processes inpuZts using Gemini 1.5 Flash Pro to:
(User-frontend(user information)-backend(receive information and process)-Gemini AI(generate recommendation)-Recommendation logic-Frontend(result)-Output)
a



Pre-requisites

IBM Cloud Account Setup: IBM Cloud Console
Watsonx Access and Configuration: Watsonx Overview
Python Basics: Python.org
FastAPI Framework: FastAPI Docs
(What are the things needed for the project)

Project Workflow(setup AI connection)

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.

Activity 1.2: Configure API keys, usage limits, and test integration for multimodal support (text + image).

Activity 1.3: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development(Built Home,ooooooom,mnjkjbkjParty,Jewelry logic)

Activity 2.1: Design Flask-based backend structure for routing, session control, and planner modules.

Activity 2.2: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.

Activity 2.3: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)(Fast API backends and routes)
Activity 3.1: Define Flask routes like /generate-home, /generate-party, /generate-jewelry.

Activity 3.2: Organize backend using modular structure:

routes/: Endpoints
services/: Recommendation logic & AI calls
models/: Input/output schemas

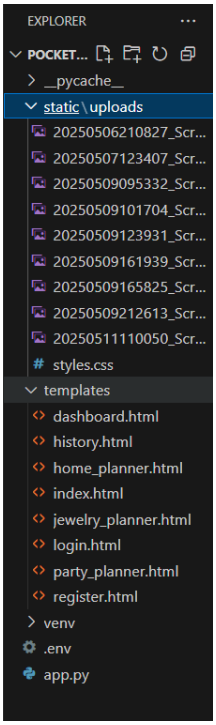
Activity 3.3: Setup CORS headers for frontend communication and implement basic session management.

Activity 3.4: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development(To create frontend page)

Activity 4.1: Build responsive HTML forms for:

Home Interior Planner (room details, quantities)
Party Planner (event type, budget, guests)



Jewelry Planner (budget, occasion, outfit image upload)

Activity 4.2: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization(To cross check the application we created)

Activity 5.1: Perform testing with real-world use cases across all three planners.

Activity 5.2: Evaluate Gemini response quality, budget adherence, and platform accuracy.

Activity 5.3: Optimize prompts and add validations to prevent input edge-case failures.

Activity 5.4: Add fallback/default recommendations when AI returns insufficient results.

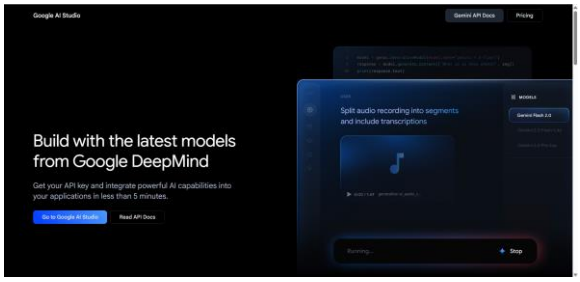
Fast API Initialization

MILESTONE 1: Gemini AI Initialization

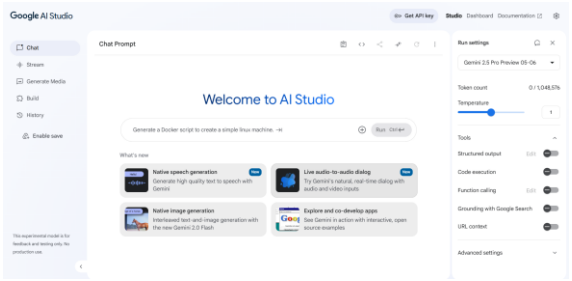
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power PocketSmart AI. The platform relies on Gemini 1.5 Flash Pro—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner).Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

- Activity 1.1: Set up Google Cloud account and enable Gemini API

Sign in or create a Google Cloud account via console.cloud.google.coconsolem.
i



Accept all terms and conditions and click on Get API key

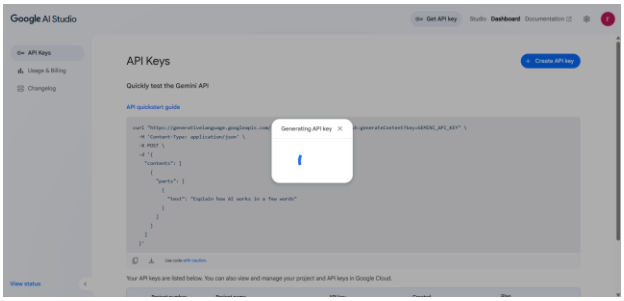


You will get trending models here you can select the model here or you can set the model by going togo to the API Documentation.
Activity 1.2: Configure access and permissions

You will get interface like this and now click on Create API key



Once click on API key you will the api key and make sure to copy the API key



Activity 1.3: Validate Gemini API connectivity

Make a sample prompt call using Python to confirm integration.



Test both text-only and image + text prompts to ensure multimodal compatibility.



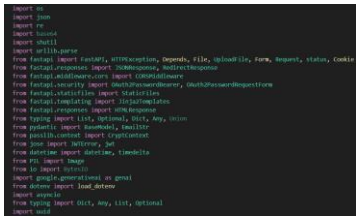
Set Google cloud access
Gemini API configure
Create API key
Using API key in python we send sample to gemini
Test text,images
Api key-Authendication needed for gemini ai to access the application. do not make it public in github..env file-has configuration and sensitive value.

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence of PocketSmart AI using the Flask framework and Google's Gemini 1.5 Flash Pro foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.
File Explorer Structurej

Activity 2.1: Fast API Instilization and import libraries

Import all required libraries for the project



Fast API Initialization
‘

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

Load .env environment



Activity 2.2: Main Functionality code routings

/generate-home: Generates home interior recommendations (furniture, decor, lighting)

```python
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

get home_recommendations by using the function



linking the websites like amazon,flipkar,ikea ect…. to get the products



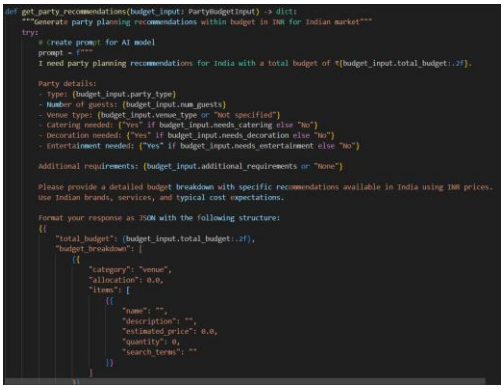
```python
    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")

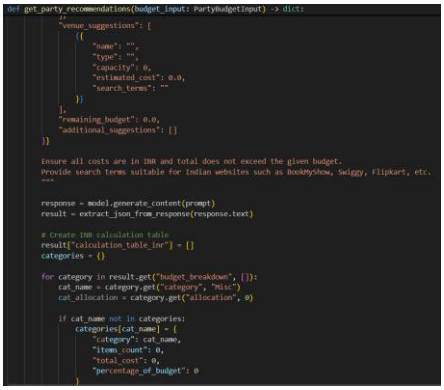

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

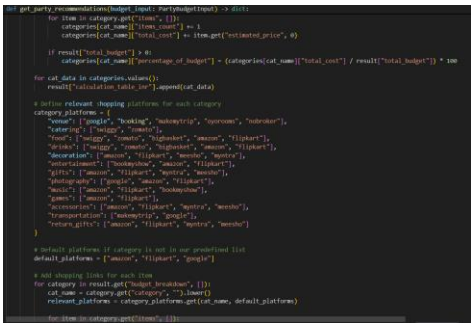
/generate-party: Suggests catering, venue, and decoration options for events

```python
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

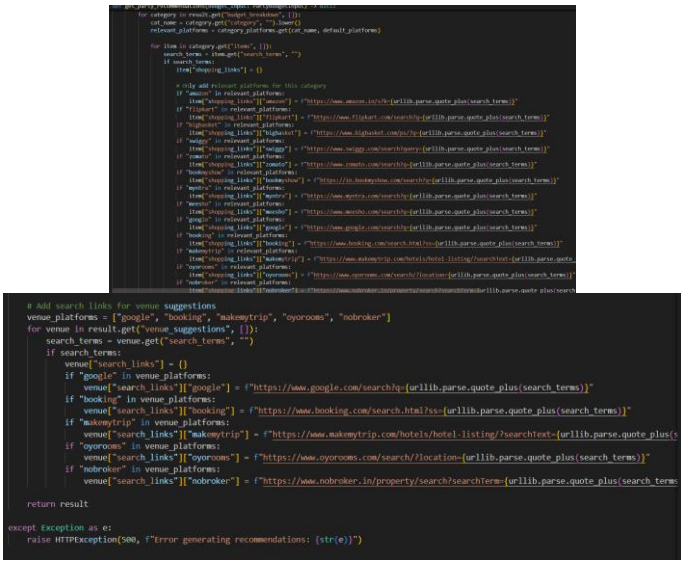
get home_recommendations by @using the function







using the if condition to categorize the situation based products



/generate-jewelry: Accepts text + image inputs to return style-matched jewelry options.
async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
"""Jewelry budget planner page"""
return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})
get home_recommendations by usicfbfxfng the function

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
                "styling_tips": []
            }

            Make sure prices are in INR and stay within budget.
            Include Indian-friendly search terms for shopping.
            """
            response = model.generate_content([prompt, img])
        else:
            # Only text input
            prompt = base_prompt + """

            Format the output as JSON:
            {
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
                "styling_tips": []
            }

            Keep prices in INR and relevant to Indian brands.
            """
            response = model.generate_content(prompt)

        result = extract_json_from_response(response.text)

        # Add shopping links for each item (India-specific)
        for item in result.get("jewelry_recommendations", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "blackstone": f"https://www.blackstone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                    "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                    "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                }

        return result

    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Activity 2.3: Login and Register page routings

Login page
/login: Authenticates user credentials and initiates a session upon successful login.

```python
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

Register Page
/register: Handles new user registration by accepting and securely storing user credentials.

```python
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

Logout Page
/logout: Terminates the current user session and redirects to the login page.

```python
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTError:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")
```

Activity 2.4: Ensure modular code structure

/token: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.



/session-info: Retrieves metadata about the current user session, such as user ID and login status.
/session-data: Returns detailed session-specific data used for personalization and recommendation tracking.

```python
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60,  # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

Built actual project functionality
Home,Party,jewelry-each have separate logic
Codes are given
Routings-What can execute according to the request
Login page-Existing user
Register page-New user
Logout page-End the current session
JWT token-We can access protected APLs(authorised access)

MILESTONE 3: Backend – FastAPI Integration (main.py)

This milestone focuses on implementing the FastAPI-powered backend for PocketSmart AI. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in gemini_utils.py. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains.It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

Activity 3.1: Define FastAPI Routes

Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions.

```python
@app.post("/home-budget")
async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_home_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

```python
# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.



```
    return {"history": []}

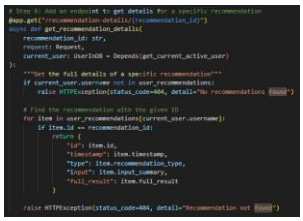
# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.
Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.
Activity 3.2: Modular Architecture Setup

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

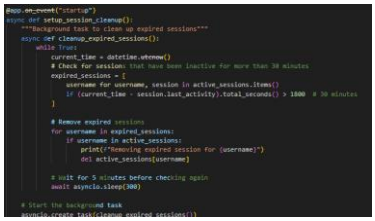


Activity 3.3: CORS & Static Routing

/history: Retrieves the user's past recommendation queries and results for review or re-use.

```
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

Activity 3.4: Startup and main function/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.



__main__: Entry point of the FastAPI application that starts the server using uvicorn when the script is run directly.

```python
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

To build complete backend structure
Fast APIs is a bridge between frontend and gemini ai
to receive request
input validate and process
communicate with gemini
return response
handles authentication and sessions
Fast API- python web framework for building APIs
route-function-processing-response
instead of single file seperate into modules
routes-handles API end points
services-business logic and gemini calls
models-defines input output data structure
CORS-helps frontend and backend safe communication

MILESTONE 4: UI Development

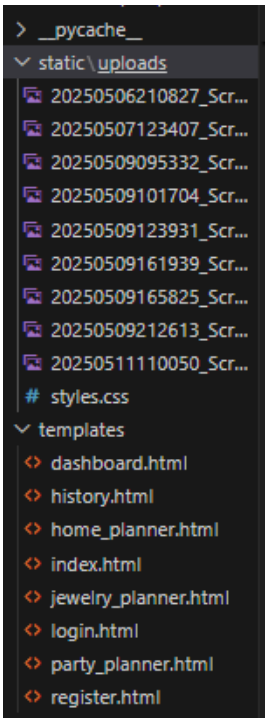
This milestone focuses on developing a lightweight, responsive, and intuitive frontend for PocketSmart AI using HTML, CSS, and Jinja2 templates. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. .5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine experience that bridges multiple categories and e-commerce ecosystems.

Scenario 1: HoScenario me Interior Planning with Smart Budget Allocation

htly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering.The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
Implement individual forms for each planner module connected to FastAPI POST routes
Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)
Templates Directory Structure:
built frontend
technologies UI
creates User friendly interface

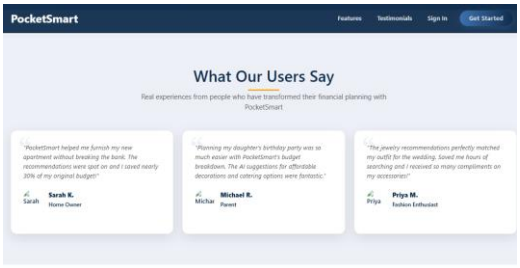


MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.

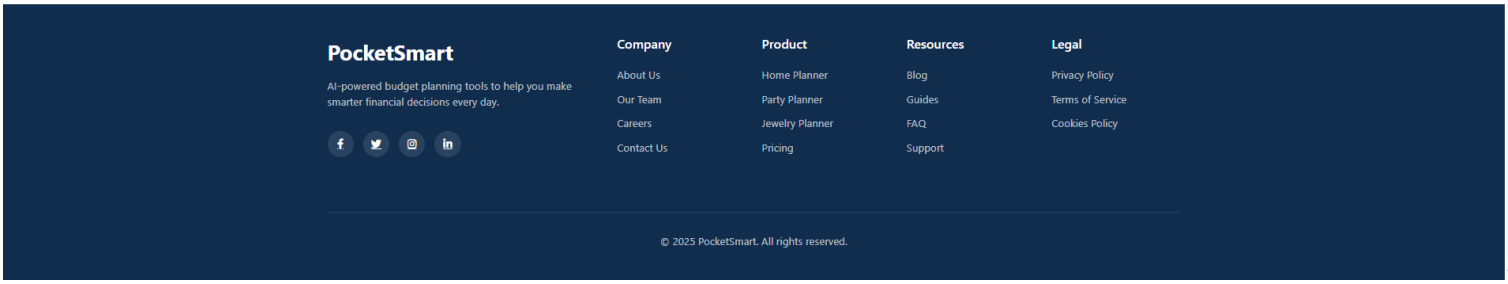
Activity 5.1: Test with Real-World Budget Inputs Across Domains Conduct end-to-end testing using varied budgets and user preferences for:

Home decor setups from platforms like IKEA, Amazon
Party packages including food (Swiggy/Zomato), venues (OYO), and decor
Jewelry suggestions matched with outfit styles, occasions, and image inputs
demonstrate how we can conbine gemini at,APIs, backend development,frontend development
to solve a practical real world budgeting recommendation problem
Home Page
Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.
Testimonials Page
Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform's effectiveness.



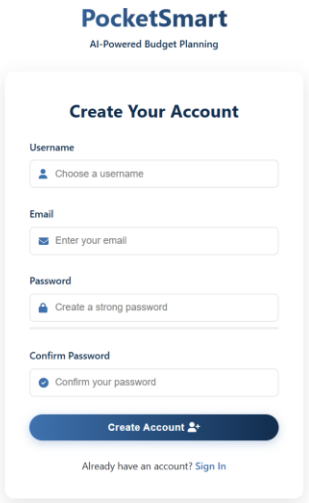


Footer Part

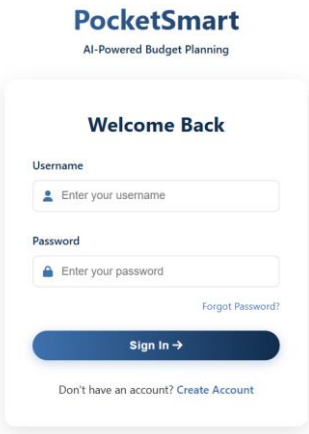


Register Page

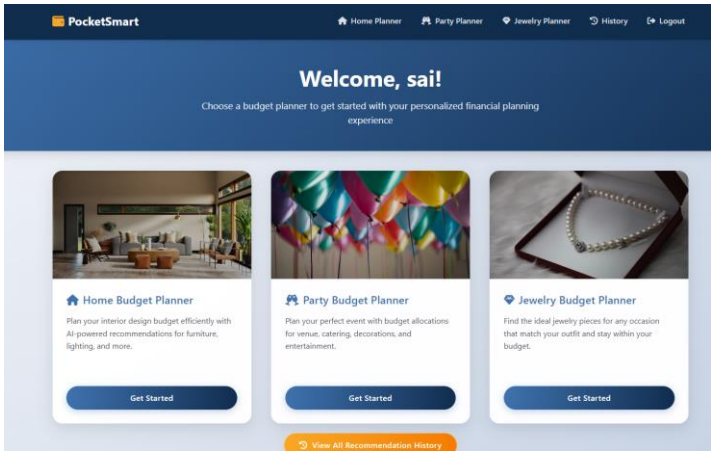
Register Page: Allaows new users to create an account to access personalized budget recommendations.
abirami

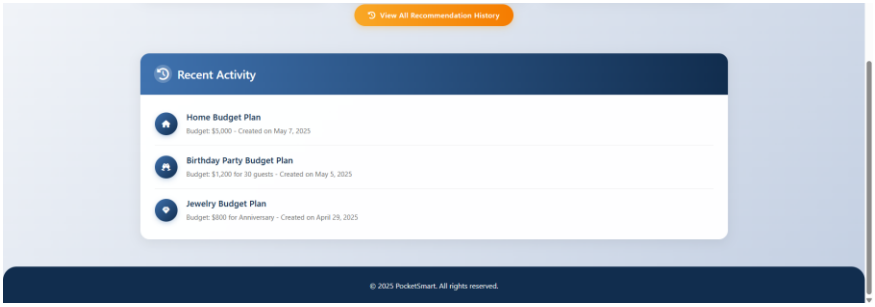


Login Page
Login Page: Authenticates existing users to access their dashboard and saved data.

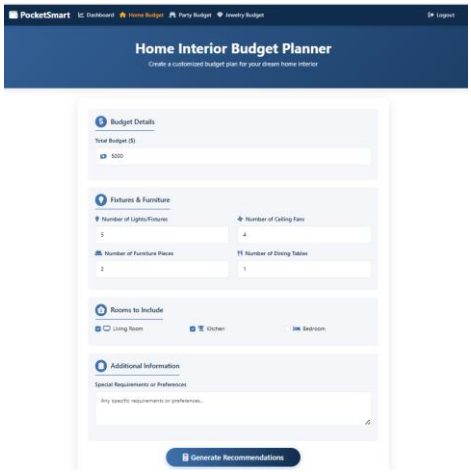


kkkkkkkkkkbvnk
User Dashboard
User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.

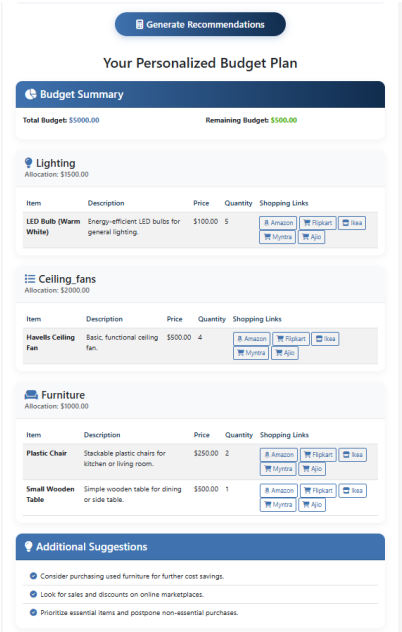




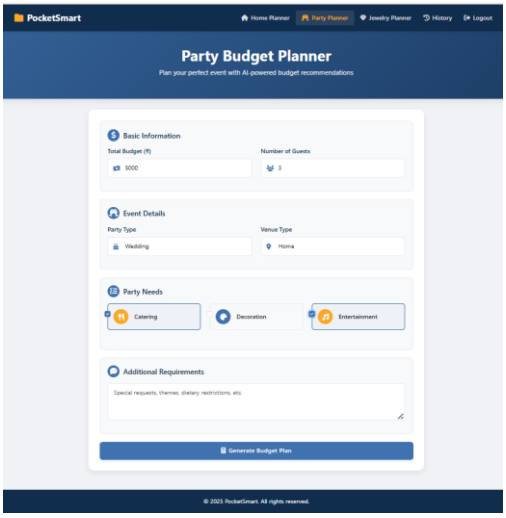
Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.



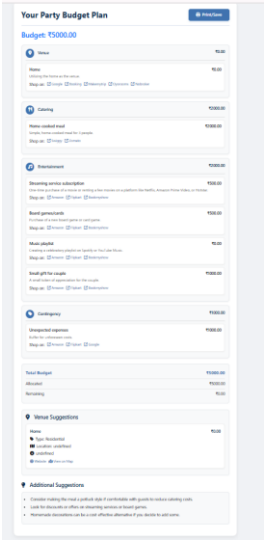
Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.



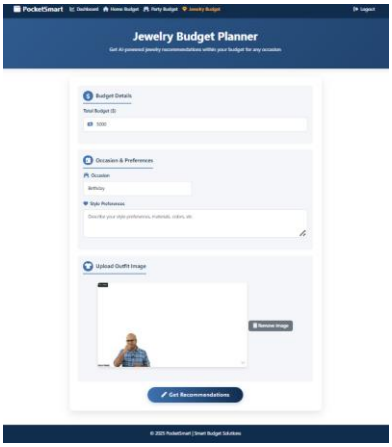
Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count



Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.



Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.



Jewelry Budget Recommendations Page: Presents personalized jewelry opg/,mo b bnk0[-- n-PocketSmart AI: Your Smart Budget & Recommendation Assistant(Gen AI powered budget
-.and recommendation assistant)

Project Description

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the Home Budget Planner. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon.Recommendations are balanced across functionalitytfunctionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

Scenario 2: AI-Based Party Budget Planning

Scenario 3: Jewelry Recommendations for Occasions

(In this project we are gonna see about home,party and jewelry budget planner and ai gives recommendation)
Act as an experienced Generative AI Project Developer and carefully analyze the provided project documentation.
Build the complete project from scratch, including all required folders, files, dependencies, and configurations.
Generate full, working code for every file, not just the partial snippets provided in the documentation.
Ensure all frontend, backend, APIs, AI/LLM integration, and configurations work together without errors.
Finally, provide simple VS Code setup, installation, running, and testing instructions for the complete application.

Architecture Overview

Core Technologies
Flask: Backend service layer for request handling, routing, and session control.
Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.): Product and service data sourcing.
Frontend (HTML/CSS + JS): Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component
Description
Frontend UI

Flask Backend
Gemini AI Layer
Processes inpuZts using Gemini 1.5 Flash Pro to:
a
Pre-requisites
IBM Cloud Account Setup: IBM Cloud Console
Watsonx Access and Configuration: Watsonx Overview
Python Basics: Python.org
FastAPI Framework: FastAPI Docs
(What are the things needed for the project)

Project Workflow(setup AI connection)

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.

Activity 1.2: Configure API keys, usage limits, and test integration for multimodal support (text + image).

Activity 1.3: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development(Built Home,Party,Jewelry logic)

Activity 2.1: Design Flask-based backend structure for routing, session control, and planner modules.

Activity 2.2: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.

Activity 2.3: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)(Fast API backends and routes)

Activity 3.1: Define Flask routes like /generate-home, /generate-party, /generate-jewelry.

Activity 3.2: Organize backend using modular structure:

routes/: Endpoints
services/: Recommendation logic & AI calls
models/: Input/output schemas

Activity 3.3: Setup CORS headers for frontend communication and implement basic session management.

Activity 3.4: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development(To create frontend page)

Activity 4.1: Build responsive HTML forms for:

Home Interior Planner (room details, quantities)
Party Planner (event type, budget, guests)
Jewelry Planner (budget, occasion, outfit image upload)

Activity 4.2: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization(To cross check the application we created)

Activity 5.1: Perform testing with real-world use cases across all three planners.

Activity 5.2: Evaluate Gemini response quality, budget adherence, and platform accuracy.

Activity 5.3: Optimize prompts and add validations to prevent input edge-case failures.

Activity 5.4: Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization

- Activity 1.1: Set up Google Cloud account and enable Gemini API

Sign in or create a Google Cloud account via console.cloud.google.coconsolem.
i
Accept all terms and conditions and click on Get API key

Activity 1.2: Configure access and permissions

You will get interface like this and now click on Create API key
Once click on API key you will the api key and make sure to copy the API key
Activity 1.3: Validate Gemini API connectivity

Make a sample prompt call using Python to confirm integration.
Test both text-only and image + text prompts to ensure multimodal compatibility.
Set Google cloud access
Gemini API configure
Create API key
Using API key in python we send sample to gemini
Test text,images

MILESTONE 2: Core Functionalities Development

File Explorer Structure

Activity 2.1: Fast API Instilization and import libraries

Import all required libraries for the project
Fast API Initialization
‘
Load .env environment

Activity 2.2: Main Functionality code routings

/generate-home: Generates home interior recommendations (furniture, decor, lighting)
get home_recommendations by using the function
linking the websites like amazon,flipkar,ikea ect…. to get the products
m m
/generate-party: Suggests catering, venue, and decoration options for events
get home_recommendations by using the function
using the if condition to categorize the situation based products
/generate-jewelry: Accepts text + image inputs to return style-matched jewelry options.
async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
"""Jewelry budget planner page"""
return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})
get home_recommendations by using the function

Activity 2.3: Login and Register page routings

Login page
/login: Authenticates user credentials and initiates a session upon successful login.
Register Page

/register: Handles new user registration by accepting and securely storing user credentials.
Logout Page
/logout: Terminates the current user session and redirects to the login page.

Activity 2.4: Ensure modular code structure

/token: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.
/session-info: Retrieves metadata about the current user session, such as user ID and login status.
/session-data: Returns detailed session-specific data used for personalization and recommendation tracking.
Built actual project functionality
Home,Party,jewelry-each have separate logic
Codes are given
Routings-What can execute according to the request
Login page-Existing user
Register page-New user
Logout page-End the current session
JWT token-We can access protected APLs(authorised access)

MILESTONE 3: Backend – FastAPI Integration (main.py)

Activity 3.1: Define FastAPI Routes

Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions.
Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.
User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.
Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.
Activity 3.2: Modular Architecture Setup

Activity 3.3: CORS & Static Routing

/history: Retrieves the user's past recommendation queries and results for review or re-use.
__main__: Entry point of the FastAPI application that starts the server using uvicorn when the script is run directly.
To build complete backend structure
Fast APIs is a bridge between frontend and gemini ai
to receive request
input validate and process
communicate with gemini
return response
handles authentication and sessions
Fast API- python web framework for building APIs
route-function-processing-response
instead of single file seperate into modules
routes-handles API end points
services-business logic and gemini calls
models-defines input output data structure
CORS-helps frontend and backend safe communication

MILESTONE 4: UI Development

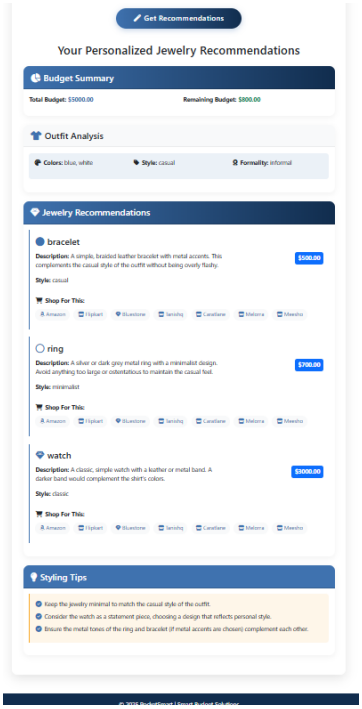
This milestone focuses on developing a lightweight, responsive, and intuitive frontend for PocketSmart AI using HTML, CSS, and Jinja2 templates. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering.The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

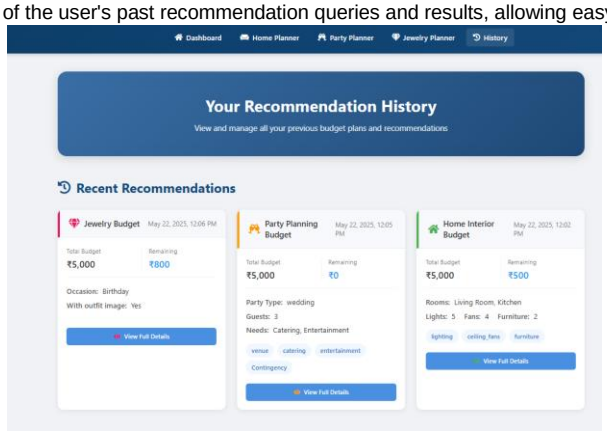
Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
Implement individual forms for each planner module connected to FastAPI POST routes
Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)
Templates Directory Structure:
built frontend
technologies UI
creates User friendly interface

MILESTONE 5: Testing & Optimization

Home decor setups from platforms like IKEA, Amazon
Party packages including food (Swiggy/Zomato), venues (OYO), and decor
Jewelry suggestions matched with outfit styles, occasions, and image inputs
demonstrate how we can conbine gemini at,APIs, backend development,frontend development
to solve a practical real world budgeting recommendation problem
Home Page
Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.
tions tailored to the occasion, outfit, and budget, using Gemini-generated insights.



Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.





Conclusion

http://console.cloud.google.com PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across home interiors, party planning, and jewelry selection. By combining the power of FastAPI with Gemini 1.5 Flash Pro, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO and more.From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on efficiency, scalability, and user-centric design. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it's finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a smart digital assistant, saving time and effort while optimizing outcomes.With its seamless fusion of generative AI and web technologies, PocketSmart AI stands as a practical example of how AI can enhance everyday decision-making, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized..ss

ts